Introduction

Python is a high-level, general-purpose programming language. This tutorial walks through data types, control flow, functions, modules and a few classic algorithms, with runnable examples in every chapter.

What you will learn

· Data types and variables
· Control flow with if, for and while
· Writing and calling functions
· Importing modules from the standard library
· Searching and sorting algorithms and their complexity

Installing Python

```
# check the installed version
python3 --version

# create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate
```

Data types

Strings

Strings are immutable sequences of characters. Use single or double quotes, and triple quotes for multi-line text. Slicing returns a new string and never modifies the original, so s[0:3] gives the first three characters.

Lists

Lists are ordered, mutable collections that can hold values of any type. Note: Python uses zero-based indexing for lists, so the first element is items[0] and the last is items[-1].

Note: Python uses zero-based indexing for lists.

Numeric types

Type	Example	Mutable	Notes
int	42	no	Arbitrary precision
float	3.14	no	64-bit IEEE 754
complex	1+2j	no	Real and imaginary parts
bool	True	no	Subclass of int

List methods

Method	What it does	Time
append(x)	Adds x to the end	O(1)

(c) 2026 Example Corp. All rights reserved.

insert(i, x)	Inserts x at index i	O(n)
pop()	Removes and returns the last item	O(1)
sort()	Sorts the list in place	O(n log n)
index(x)	Returns the position of x	O(n)

Control flow

if statements

```
temperature = 31

if temperature > 30:
    print('Hot day')
elif temperature > 20:
    print('Warm day')
else:
    print('Cool day')
```

for loops

```
scores = {'ana': 91, 'ben': 78, 'chen': 85}

for name, score in scores.items():
    if score >= 80:
        print(f'{name} passed with {score}')
```

while loops

A while loop repeats as long as its condition is true. Use break to leave the loop early and continue to skip to the next iteration. Always make sure something inside the loop changes the condition, or it will run forever.

Exercises

Each chapter ends with short exercises. Click here to download the full source code for this chapter. Try to solve each exercise before you look at the solution file.

Functions

Defining functions

```
def greet(name, greeting='Hello'):
    """Return a greeting for name."""
    return f'{greeting}, {name}!'

print(greet('world'))
print(greet('Ada', greeting='Hi'))
```

Default arguments

Default values are evaluated once, when the function is defined. Never use a mutable default such as an empty list; use None and create the list inside the function instead.

(c) 2026 Example Corp. All rights reserved.

Kinds of arguments

1. Positional arguments are matched by order.
2. Keyword arguments are matched by name.
3. *args collects extra positional arguments into a tuple.
4. **kwargs collects extra keyword arguments into a dict.

Modules

Importing modules

```
import math
from collections import Counter

print(math.sqrt(16))
words = 'the cat and the hat'.split()
print(Counter(words).most_common(1))
```

Standard library highlights

Module	Use it for
os	Files, folders and environment variables
json	Reading and writing JSON
datetime	Dates, times and time zones
re	Regular expressions
itertools	Fast looping building blocks

Algorithms

Search

Linear search checks every element in turn and takes O(n) time. When the data is sorted, binary search is much faster because it discards half of the remaining range at every step.

Binary Search

```
def binary_search(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Binary search runs in O(log n) time on a sorted list.

Click here to download the full source code for this chapter.

Sorting

(c) 2026 Example Corp. All rights reserved.

Python's built-in sorted() uses Timsort, a stable hybrid of merge sort and insertion sort. It returns a new list, while list.sort() sorts in place and returns None.

Merge sort

```
def merge_sort(items):
    if len(items) <= 1:
        return items
    mid = len(items) // 2
    left = merge_sort(items[:mid])
    right = merge_sort(items[mid:])
    merged = []
    while left and right:
        merged.append(left.pop(0) if left[0] <= right[0] else right.pop(0))
    return merged + left + right
```

Comparison of sorting algorithms

Algorithm	Best	Average	Worst	Stable
Bubble sort	O(n)	O(n^2)	O(n^2)	yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	yes
Quick sort	O(n log n)	O(n log n)	O(n^2)	no
Timsort	O(n)	O(n log n)	O(n log n)	yes

Complexity

· Linear search: O(n)
· Binary search: O(log n)
· Merge sort: O(n log n)
· Dictionary lookup: O(1) on average

Errors

Handling errors

```
try:
    value = int(input('Enter a number: '))
except ValueError:
    print('That was not a number.')
else:
    print(f'You entered {value}')
finally:
    print('Done.')
```

Dictionaries and sets

Dictionaries

A dictionary maps keys to values. Keys must be hashable, which means immutable types such as strings, numbers and tuples. Looking up a key takes O(1) time on average, which makes dictionaries the right choice for counting and caching.

```
inventory = {"apples": 12, "pears": 4}
```

(c) 2026 Example Corp. All rights reserved.

```
inventory["plums"] = 7

for fruit, count in sorted(inventory.items()):
    print(f"{fruit:>8}: {count}")

total = sum(inventory.values())
print("total", total)
```

Dictionary methods

Method	Returns	Notes
get(key, default)	The value or the default	Never raises KeyError
setdefault(key, value)	The existing or new value	Inserts when missing
items()	A view of (key, value) pairs	Reflects later changes
pop(key)	The removed value	Raises KeyError if missing
update(other)	None	Overwrites existing keys

Sets

A set holds unique items with no order. Use a set to remove duplicates from a list or to test membership quickly. Sets support union, intersection and difference with the operators |, & and -.

Exercises

Try each exercise before you open the solution file. Run your code after every small change so mistakes are easy to find.

Classes

Defining a class

A class bundles data with the functions that work on it. The `__init__` method runs when a new object is created, and `self` refers to that object.

```
class BankAccount:
    """A minimal account with deposits and withdrawals."""

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("insufficient funds")
        self.balance -= amount
        return self.balance

    def __repr__(self):
```

(c) 2026 Example Corp. All rights reserved.

```
        return f"BankAccount({self.owner!r}, {self.balance})"
```

Inheritance

A subclass reuses everything from its parent and can override methods. Call `super().__init__()` so the parent sets up its own attributes before the subclass adds more.

Dunder methods

· `__init__`: sets up a new object.
· `__repr__`: the developer-friendly text shown in the console.
· `__str__`: the user-friendly text used by print.
· `__eq__`: decides when two objects count as equal.
· `__len__`: makes len(obj) work.
· `__iter__`: makes the object usable in a for loop.

Exercises

Try each exercise before you open the solution file. Run your code after every small change so mistakes are easy to find.

Working with files

Reading and writing text

Always open files with a `with` block so they are closed even if an error happens. Pass `encoding="utf-8"` explicitly; the default depends on the operating system.

```
from pathlib import Path

notes = Path("notes.txt")
notes.write_text("first line\nsecond line\n", encoding="utf-8")

with notes.open(encoding="utf-8") as handle:
    for number, line in enumerate(handle, start=1):
        print(number, line.rstrip())
```

File modes

Mode	Meaning	Creates the file	Truncates
r	Read text	no	no
w	Write text	yes	yes
a	Append text	yes	no
rb	Read bytes	no	no
x	Create, fail if it exists	yes	no

JSON files

The json module converts between Python objects and JSON text. Use `json.dump` to write a file and `json.load` to read one; both take an open file object.

(c) 2026 Example Corp. All rights reserved.

Testing

Writing tests with pytest

A test is a function whose name starts with `test_` and that uses plain assert statements. pytest finds these functions automatically and shows a readable difference when an assertion fails.

```
import pytest

from bank import BankAccount


def test_deposit_increases_balance():
    account = BankAccount("ada")
    assert account.deposit(50) == 50


def test_withdraw_more_than_balance_fails():
    account = BankAccount("ada", balance=10)
    with pytest.raises(ValueError):
        account.withdraw(20)
```

What to test

1. The normal case with typical input.
2. Edge cases: empty input, zero, the largest allowed value.
3. Error cases: the function raises the right exception.
4. Anything that broke before, so it cannot break again.

Summary

· Strings and numbers are immutable; lists and dicts are mutable.
· Prefer for loops over while loops when you know what you are iterating over.
· Use binary search on sorted data and sorted() when you need a new sorted list.
· Catch the specific exception you expect, never a bare except.

Recap of searching

Linear search checks every element in turn and takes O(n) time. When the data is sorted, binary search is much faster because it discards half of the remaining range at every step.

(c) 2026 Example Corp. All rights reserved.